

From algebraic roots to radicals

A branch-aware solver based on structural reductions
and constructive Galois theory

Technical report prepared for Vladimir Reshetnikov

8 September 2026 • Reference implementation 0.1.0

Abstract

We construct a radical-conversion system for exact algebraic numbers represented by polynomial roots. The starting point is the supplied `FindExtension.nb` notebook and its searches for coefficient fields of quadratic and cubic factors. The replacement has two layers. Exact structural reductions produce short formulas for power compositions, Dickson polynomials, and generalized reciprocal polynomials; in particular, they solve both examples in the original question. A general reference engine constructs a splitting field, recovers its actual automorphisms, and performs Fourier resolvent descent along a prime-index subnormal series. Roots of unity are handled in a formal cyclotomic tensor algebra, so no linear-disjointness assumption is needed. A norm polynomial supplies the hypothesis needed for branch comparisons based on root separation. We prove finite completeness of the mathematical algorithm, explain the boundary between that theorem and practical resource limits, and document executable Python code and a Wolfram Language frontend with native structural methods. The Python implementation is tested; the supplied Wolfram code has not been executed in a Wolfram kernel in this environment.

Scope. Parameter-free algebraic numbers over $\mathbb{Q}$, with complex radicals allowed. No claim of minimal radical depth, shortest formula, or practical completion for every large solvable input is made.

Contents

1	The problem and the contract	4
1.1	Three different questions	4
1.2	Allowed output and input	4
1.3	What “complete” means here	5
2	What the supplied notebook contributes	5
2.1	Factor coefficients are resolvents in disguise	5
2.2	Projections and experimental recognition	5
2.3	Engineering changes	6
3	Structural reductions before Galois closure	6
3.1	Translation and power composition	6
3.2	Dickson polynomials and correlated branches	7
3.3	Generalized reciprocal composition	7
4	The two original examples, explicitly	8
4.1	The sextic: a cubic followed by a quadratic	8
4.2	The quintic: a Dickson equation	8
5	Constructing the exact normal closure	9
5.1	Adjoin a root of a remaining factor	9
5.2	Why embedding changes are harmless at this stage	10
5.3	Termination of the closure construction	10
5.4	A worked closure: $x^3 - 2$	10
6	Recovering the actual automorphisms	10
6.1	Weighted conjugate sums instead of a second hard factorization	11
6.2	The multiplication table	11
7	From a solvable group to prime cyclic steps	11
8	Fourier descent without a disjointness assumption	12
8.1	The tensor algebra	12
8.2	The resolvent identities	13
8.3	Recursive conversion	13
8.4	Zero resolvents are not exceptions to the theory	14

9	Branch selection using an exact annihilator	14
9.1	The norm polynomial	14
9.2	Equality is easier when a polynomial is already known	14
9.3	Two levels of verification	15
10	Correctness and finite completeness	15
11	Implementation details that affect correctness	16
11.1	Rational vectors and orientation	16
11.2	No arbitrary tensor division	16
11.3	Expression DAGs, not executable source strings	16
11.4	Root indices are not portable	17
11.5	Failure and time-limit semantics	17
12	Using the package	18
12.1	Wolfram Language	18
12.2	Python	18
13	Executed tests and observed limitations	19
13.1	Environment and reproducibility	19
13.2	Coverage	19
13.3	What the stress run did and did not establish	20
14	Complexity and further improvements	20
14.1	The main costs	20
14.2	Improvements that preserve the proof structure	21
15	Conclusion	21
A	Compact algorithm descriptions	22
A.1	Normal closure and automorphisms	22
A.2	One radical-conversion call	22
B	Files and rebuilding	23

1 The problem and the contract

A `Root` expression is an excellent exact representation of an algebraic number, but it deliberately hides how that number was obtained. The conversion problem is to recover a finite expression involving rational arithmetic and rational powers, with the *same complex embedding* as the input. The official `ToRadicals` documentation explicitly covers all degrees through four and notes that some higher-degree radical-solvable cases are missed [5]. Failure of that transformation is therefore not a Galois-theoretic impossibility result.

The motivating question supplies, among other examples,

$$r_6 = \text{Root}[-1 - \#^2 - \#^3 + \#^4 + \#^6 \&, 2], \quad (1)$$

$$r_5 = \text{Root}[6 + 25\# - 25\#^3 + 5\#^5 \&, 5]. \quad (2)$$

The examples and notebook originate with Reshetnikov [1, 2]. The package treats this as an algorithm-construction task, not as a request to find a convenient expression for just one root.

1.1 Three different questions

For an input α , three tasks must remain separate: *decide* whether radicals exist, *construct* such radicals, and *identify* the requested conjugate. A correct abstract Galois group can settle the first question while providing no formulas for the second. An annihilating polynomial can settle neither the second nor the third by itself: $\sqrt{2}$ and $-\sqrt{2}$ have the same minimal polynomial.

Let f be the irreducible minimal polynomial of α over $\mathbb{Q}$, and let L be its splitting field. The classical criterion is

$$\alpha \text{ is expressible by radicals over } \mathbb{Q} \iff \text{Gal}(L/\mathbb{Q}) \text{ is solvable.} \quad (3)$$

This is a statement about the *minimal polynomial of the selected number*, not an arbitrary reducible polynomial that happens to vanish there. The classical cyclic-extension and solvability theorems are described in Milne's account [3]; the explicit finite construction below gives the sufficiency direction in the form needed by this implementation.

1.2 Allowed output and input

The accepted output grammar is generated by rational constants, i , addition, multiplication, and b^q for $q \in \mathbb{Q}$, interpreted using principal complex powers. Integer negative powers include division by nonzero expressions. Explicit undefined values are rejected. In particular,

$$\zeta_n = (-1)^{2/n} \quad (4)$$

is an allowed radical, whereas an unevaluated trigonometric function or `Root` is not. Intermediate `Root/CRootOf` objects are allowed inside the algorithm; none may escape into its radical result.

The Python interface takes a rational polynomial and a zero-based SymPy root index. The Wolfram interface takes an exact algebraic expression, obtains its minimal polynomial, and preserves the original embedding by a final exact comparison. Approximate coefficients and free parameters are rejected. A symbolic parameter introduces specialization and branch-region questions that are outside this version's contract. An algebraic number whose defining expression happens to use algebraic coefficients is still in scope when its rational minimal polynomial can be obtained.

1.3 What “complete” means here

The general algorithm is not a bounded search over guessed radicals. It is a finite construction using polynomial factorization, resultants, finite groups, and exact algebraic embedding comparisons. Removing degree and time limits removes the implementation’s deliberate cutoffs. The completeness theorem assumes these standard exact primitives return correctly; it is not a machine-checked proof of the Python program or a guarantee about the running time of a particular CAS backend. This distinction matters particularly for splitting fields and reconstruction of minimal polynomials from large radical formulas.

The delivered Wolfram file contains the structural algorithms natively and calls the Python general engine when required. Thus the *general theorem* applies to an implemented engine, not to a placeholder for a future Galois routine. The Wolfram kernel execution status is stated separately in Section 13.

2 What the supplied notebook contributes

The notebook was inspected as data, without evaluating its cells. A readable, best-effort box extraction is included beside the unchanged notebook. That extraction is not a runnable Wolfram program and is not a replacement for the original source.

2.1 Factor coefficients are resolvents in disguise

The notebook’s `findExtension` imposes divisibility by

$$x^2 + px + q. \tag{5}$$

Equating the two coefficients of the remainder to zero and eliminating q produces candidate values of p . If the quadratic’s roots are α_i and α_j , then

$$p = -(\alpha_i + \alpha_j), \quad q = \alpha_i \alpha_j. \tag{6}$$

Thus its coefficient search is already an implicit search among symmetric functions of subsets of conjugates. The analogous cubic search uses elementary symmetric functions of triples. This is a natural practical way to discover a smaller field in which the original polynomial factors.

What it does not provide is a completeness argument. A useful fixed field need not be generated by the particular coefficient selected for elimination. Restricting factor degrees to two and three does not cover a cyclic quotient of prime order five or seven. Even a suitable factorization does not by itself make all coefficient-field elements radical expressions over $\mathbb{Q}$.

The filter involving `ExactNumberQ` discards explicitly rational or rational-complex numeric coefficients; that predicate is not a general test for algebraicity [6]. Discarding a rational trace can also conceal a useful extension generated by a different factor coefficient. The new system uses exact field coordinates rather than the printed form of a candidate coefficient as its structural invariant.

2.2 Projections and experimental recognition

The notebook’s `project` looks at real and imaginary parts of conjugates. Those can indeed reveal smaller fields. The `fuzz` routine takes random subsets of roots, approximates their sums, applies

RootApproximant, and then tries **PossibleZeroQ**. This is useful discovery machinery, but an integer-relation or algebraic-recognition guess is not a proof that it equals the original sum. It must be followed by a separate exact check.

The notebook also contains repeated interactive factor/solve experiments, large polynomials associated with inverse-beta examples, references to global state and earlier outputs, and iterative simplification. Such a research notebook should not be repackaged by simply exporting all definitions and calling the result a complete solver.

2.3 Engineering changes

The replacement makes the trusted conditions explicit. Polynomial identities are checked before structural reconstruction; the normal closure supplies the actual automorphisms used in general descent; and a radical candidate is never selected merely because its decimal approximation is close. Empty searches, invalid input, backend failures, resource cutoffs, and proved nonsolvability have different results. Public code does not depend on the private **Simplify** context or on notebook output history.

The notebook's **radicalDepth** also counts trigonometric forms for its complexity heuristic. That is a useful aesthetic measure, but it should not be confused with the literal radical grammar required here. The package supplies an explicit **RadicalExpressionQ** predicate instead.

3 Structural reductions before Galois closure

The general construction can be costly even when a short answer exists. Consequently the automatic route first searches for exact identities of a few cheap forms. These searches are not themselves complete; the general engine is the fallback.

3.1 Translation and power composition

Normalize f to be monic of degree n and remove its next-to-leading term:

$$c = -\frac{[x^{n-1}]f}{n}, \quad q(y) = f(y + c). \quad (7)$$

Let g be the gcd of the positive exponents having nonzero coefficients in q . If $g > 1$, write

$$q(y) = P(y^g). \quad (8)$$

After recursively expressing the roots v of P , every root of f has the form

$$x = c + \zeta_g^j v^{1/g}, \quad 0 \leq j < g. \quad (9)$$

The recursion lowers polynomial degree. The identity proves annihilation of all reconstructed candidates; their requested embedding is a separate choice. This covers, for example, shifted binomials such as $(x + 1)^5 + 2$.

3.2 Dickson polynomials and correlated branches

Define

$$D_0(y, a) = 2, \quad D_1(y, a) = y, \quad D_n(y, a) = yD_{n-1}(y, a) - aD_{n-2}(y, a). \quad (10)$$

Direct induction gives

$$D_n(u + a/u, a) = u^n + (a/u)^n. \quad (11)$$

For a centered polynomial, the coefficient of y^{n-2} suggests $a = -[y^{n-2}]q/n$ when $n > 2$. The implementation constructs D_n and checks whether $D_n - q$ is constant; it does not accept the coefficient test alone.

If $q(y) = D_n(y, a) - b$ and $a \neq 0$, put

$$t = \left(\frac{b + \sqrt{b^2 - 4a^n}}{2} \right)^{1/n}. \quad (12)$$

Then all roots are

$$y_j = \zeta_n^j t + \frac{a}{\zeta_n^j t}. \quad (13)$$

The selected radicand is nonzero: the two roots of $T^2 - bT + a^n$ have nonzero product. Formula (13) therefore has no hidden division by zero. When $a = 0$, the code uses the binomial formula instead.

The reciprocal term is essential. Taking independent principal n th roots of the two roots of $T^2 - bT + a^n$ can destroy the condition that their product is a . Keeping the second term as a/u carries this relation through the entire calculation. This is the simplest example of why branch correlation belongs to the algebraic representation, not to a final numerical cleanup step.

3.3 Generalized reciprocal composition

For a centered monic polynomial q of degree $2m$, search for

$$q(x) = x^m P(x + a/x), \quad a \in \mathbb{Q} \setminus \{0\}, \quad \deg P = m. \quad (14)$$

The constant coefficient forces $a^m = q(0)$. Rational m th roots of a rational number can be found exactly by testing numerator and denominator for perfect powers. There are at most two candidates for a .

For each candidate, the coefficient of x^{2m} determines the leading coefficient of P , the coefficient of x^{2m-1} determines the next one, and so on. More explicitly, subtract the appropriate multiples of $x^m(x + a/x)^j$ for $j = m, m-1, \dots, 0$. The expression is a polynomial because $j \leq m$. A zero final remainder proves (14).

Once y is a root of P , recover

$$x = \frac{y \pm \sqrt{y^2 - 4a}}{2}. \quad (15)$$

This is a rational-function decomposition rather than an ordinary polynomial composition. Searching only for $f = g \circ h$ with polynomial h would miss it. The current recognizer searches rational a after the standard centering translation; it does not claim to find every possible algebraic or Möbius change of variable.

4 The two original examples, explicitly

4.1 The sextic: a cubic followed by a quadratic

The decisive identity is

$$x^6 + x^4 - x^3 - x^2 - 1 = x^3 \left[(x - 1/x)^3 + 4(x - 1/x) - 1 \right]. \quad (16)$$

It is the reciprocal case with $a = -1$ and

$$P(y) = y^3 + 4y - 1. \quad (17)$$

Put

$$u = \left(\frac{9 + \sqrt{849}}{18} \right)^{1/3}, \quad y = u - \frac{4}{3u}. \quad (18)$$

Here $u > 0$ is the ordinary positive cube root. Its cube satisfies $t^2 - t - 64/27 = 0$, and hence direct expansion gives $y^3 + 4y = 1$. The answer for (1) is

$$r_6 = \frac{1}{2} \left(u - \frac{4}{3u} + \sqrt{\left(u - \frac{4}{3u} \right)^2 + 4} \right). \quad (19)$$

Numerically this is approximately 1.1306854454620405872.

There is no ambiguous real-root choice in this derivation. The cubic is strictly increasing and has a unique real root. The function $x - 1/x$ is strictly increasing on each real half-line and maps each half-line onto all of $\mathbb{R}$. Thus the sextic has exactly two real roots; the plus sign in (19) gives the positive one requested in the question. The two real roots have product -1 .

A transparent Wolfram form is

```
With[{u = ((9 + Sqrt[849])/18)^(1/3)},
  (u - 4/(3 u) + Sqrt[(u - 4/(3 u))^2 + 4])/2
]
```

The formula was independently checked in the Python environment by minimal- polynomial reconstruction and embedding comparison. The same Wolfram expression is supplied as an example, without claiming a Wolfram-kernel execution test.

4.2 The quintic: a Dickson equation

The normalized quintic is

$$x^5 - 5x^3 + 5x + \frac{6}{5} = D_5(x, 1) + \frac{6}{5}. \quad (20)$$

Take

$$v = \left(\frac{-3 + 4i}{5} \right)^{1/5} \quad (21)$$

with its principal power. The selected root is

$$r_5 = v + v^{-1}. \quad (22)$$

Indeed, $v^5 + v^{-5} = -6/5$ and (11) proves the equation. Since $|(-3 + 4i)/5| = 1$, the inverse term is the conjugate of v ; this is also the two-conjugate-radical expression in the original question. The result is approximately 1.8070599950854245021.

If $\theta = \arg(-3 + 4i) \in (\pi/2, \pi)$, the five reconstructed roots are $2 \cos((\theta + 2\pi j)/5)$. This observation identifies the principal choice as the largest real root. It is only an explanation of the embedding: the returned expression (22) contains no trigonometric function. The implementation enumerates all five correlated branches and checks the selected embedding.

5 Constructing the exact normal closure

The remaining sections describe the general engine. Let $f \in \mathbb{Q}[x]$ be monic, irreducible, and of degree $n > 1$. We work with fields

$$K = \mathbb{Q}[t]/(h(t)), \quad h \text{ monic and irreducible.} \quad (23)$$

Elements are rational coefficient vectors of length at most $d = \deg h$; addition and multiplication are reduced modulo h . The initial field is a root field of f .

5.1 Adjoin a root of a remaining factor

Factor f over K . If every factor is linear, K is a splitting field of f , because it was built using roots of f and contains all of them. Otherwise choose an irreducible factor

$$g(Y) = Y^e + g_{e-1}(t)Y^{e-1} + \cdots + g_0(t), \quad e > 1. \quad (24)$$

Adjoining a root β gives a field of degree de over $\mathbb{Q}$. To keep using a single-generator representation, try

$$T = t + c\beta, \quad c = 1, 2, 3, \dots, \quad (25)$$

and compute

$$H_c(T) = \text{Res}_u(h(u), c^e g((T - u)/c, u)). \quad (26)$$

The notation $g(Y, u)$ means that its coefficient vectors are read as polynomials in u .

Lemma 5.1. *If H_c is square-free of degree de , then $T = t + c\beta$ is primitive for $K(\beta)/\mathbb{Q}$, and the monic polynomial H_c is its minimal polynomial. A suitable positive integer c exists.*

Proof. There are de embeddings of the separable field $K(\beta)$ into $\mathbb{C}$. The resultant is the product of $T - \tau(t) - c\tau(\beta)$ over those embeddings. It is monic of degree de . Square-freeness says that these de values are distinct, so $t + c\beta$ has de conjugates and generates the whole field.

For two distinct embeddings τ, ρ , the equality $\tau(t) + c\tau(\beta) = \rho(t) + c\rho(\beta)$ either has no solution in c or has exactly one. It cannot hold identically, because t and β generate the field. There are finitely many pairs. Avoiding their exceptional values leaves a positive integer c . $\square$

This test prevents a common primitive-element mistake: accepting a resultant of the expected degree without checking that repeated conjugates have not collapsed the actual generated field. The number of exceptional values is at most $\binom{de}{2}$, giving a finite bound for the search in principle.

5.2 Why embedding changes are harmless at this stage

After computing H_c , the implementation constructs the next field with a chosen exact root of H_c , then factors the original f again. It need not preserve the numerical embedding of every previous generator. Any root of the proved irreducible H_c defines an isomorphic field, and the polynomial f still has rational coefficients. Splitting f is an embedding-independent property.

This is deliberately different from repeatedly calling a generic primitive- element routine on a large list of already embedded `CRootOf` expressions. Here the minimal polynomial has already been proved and is passed explicitly to the field constructor. It avoids asking the backend to rediscover it. Only after the full field is available is a concrete embedding used for radical branch selection and for matching the original requested root.

5.3 Termination of the closure construction

Every adjunction is generated by a root of f , so the resulting abstract field embeds into a splitting field of f . Each nonlinear factor adjunction increases the degree by at least a factor of two. The degree is bounded by $n!$, and after at most $n - 1$ distinct roots have been adjoined, the remaining root is determined by the trace coefficient. Consequently the construction reaches a splitting field in finitely many steps.

This gives a degree bound, not a useful worst-case speed guarantee. The coefficient heights of H_c can grow substantially. Exact factorization over a number field may dominate even when the ultimate Galois group has modest order. The implementation checks its field-degree budget before making each extension.

5.4 A worked closure: $x^3 - 2$

Start with $t^3 = 2$. Over $\mathbb{Q}(t)$,

$$x^3 - 2 = (x - t)(x^2 + tx + t^2). \quad (27)$$

For the quadratic factor, the first candidate gives

$$H_1(T) = (T^3 + 2)^2, \quad (28)$$

which correctly fails the square-free test. The second gives

$$H_2(T) = T^6 + 108. \quad (29)$$

The degree-six field splits $x^3 - 2$. The repeated polynomial at $c = 1$ has a simple interpretation: the sum of two roots is the negative of the third, so it generates only a cubic field. The weighted sum with coefficients 1, 2 separates all six embeddings.

6 Recovering the actual automorphisms

Knowing that the Galois group is isomorphic to, for example, S_3 is not enough for the next stage. We need to apply each automorphism to an element represented in the chosen field basis.

6.1 Weighted conjugate sums instead of a second hard factorization

One option is to factor the primitive polynomial h over L . That is correct, but factoring a degree- d polynomial over a degree- d field can be unnecessarily expensive. The implementation uses information already accumulated during closure construction.

The final primitive element has the form

$$t = w_1\alpha_{i_1} + \cdots + w_r\alpha_{i_r}, \quad w_1 = 1, \quad w_j \in \mathbb{Z}_{>0}, \quad (30)$$

for distinct roots of the original f . The weights are the primitive-search coefficients chosen at successive steps. Although its concrete embedding may have changed, the existence of such a representation is preserved.

Enumerate ordered tuples of r distinct original roots, form the corresponding weighted sums $b \in L$, and retain precisely those satisfying $h(b) = 0$ in $\mathbb{Q}[t]/(h)$. Deduplicate by exact coefficient vectors. Stop when d distinct values have been found.

Proposition 6.1. *The procedure recovers exactly the d images of t under $\text{Gal}(L/\mathbb{Q})$. Each retained b defines an automorphism by $A(t) \mapsto A(b)$ modulo h .*

Proof. Every embedding of L permutes the roots of f and sends (30) to one of the enumerated sums. Thus all conjugates of t occur. Conversely, $h(b) = 0$ gives an injective $\mathbb{Q}$ -homomorphism $\mathbb{Q}[t]/(h) \rightarrow L$. Its image has degree d , so it is all of L . There are exactly d distinct conjugates, which is both a completeness check and a stopping criterion. $\square$

This turns a feature of the notebook's conjugate-sum experiments into a finite, exact construction: the weights are known, the search space is explicit, and the acceptance test is a zero coefficient vector, not algebraic recognition from a decimal approximation.

6.2 The multiplication table

If b_a, b_b are the primitive images for σ_a, σ_b , then

$$(\sigma_a \circ \sigma_b)(t) = b_b(b_a(t)) \pmod{h(t)}. \quad (31)$$

The apparent reversed order is important: the outer automorphism acts on the coefficient representation of the inner image. Horner substitution constructs the product table and inverse table. The identity is the element whose image is the residue class t itself.

For a field element $A(t)$ and a subgroup H , membership in its fixed field is checked directly:

$$A \in L^H \iff A(b_\sigma(t)) \equiv A(t) \pmod{h} \quad (\sigma \in H). \quad (32)$$

No smaller fixed-field primitive polynomial is required to carry out this test. The representation may be redundant, but it is uniform and exact.

7 From a solvable group to prime cyclic steps

The descent needs a subnormal chain

$$G = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_s = 1, \quad H_{i-1}/H_i \cong C_{p_i}, \quad (33)$$

where p_i is prime. These subgroups need only be normal in their immediate predecessor, not in the whole group.

For a current subgroup H , compute its commutator subgroup $D = [H, H]$ by finite group closure. If $D = H \neq 1$, a nontrivial perfect subgroup has been found and G is not solvable. Otherwise begin with $N = D$, scan the elements $g \in H$, and replace N by $\langle N, g \rangle$ whenever that generated subgroup is still proper in H .

Lemma 7.1. *At the end of this scan, N is normal in H and H/N has prime order.*

Proof. The quotient H/D is abelian, so every subgroup containing D is normal in H . If a tested g generated all of H together with an earlier N , it still does so after N grows. Thus at the end every $g \notin N$ generates the whole quotient H/N . That finite quotient has no nontrivial proper subgroup, and is therefore cyclic of prime order. $\square$

Repeat with $H = N$. The code explicitly checks normality and prime index before using a step. Any $\sigma \in H_{i-1} \setminus H_i$ generates the quotient, and σ^{p_i} fixes every element of L^{H_i} .

For degrees through six, the implementation optionally asks SymPy's exact Galois classifier for an early negative result. Its documented degree limit is six [7]. That classifier is *not* the source of the actual automorphism action used for positive construction. Above degree six, the explicit closure and multiplication-table algorithm still supplies that action. Exhaustion of the classifier's transformation attempts falls back to the general route instead of implying nonsolvability.

8 Fourier descent without a disjointness assumption

Fix one step $N \triangleleft H$ with $H/N \cong C_p$, and let $\beta \in L^N$. Choose $\sigma \in H$ generating the quotient. Classical Lagrange resolvents diagonalize the cyclic action after a primitive p th root of unity is available. This is the computational content behind the cyclic Kummer construction [3]. Related general radical-solution algorithms are discussed by Li [4]; the presentation here focuses on exact field coordinates, a division-free tensor formulation, and explicit branch bookkeeping.

8.1 The tensor algebra

Work formally in

$$A = L[z]/(\Phi_p(z)) \cong L \otimes_{\mathbb{Q}} \mathbb{Q}(\zeta_p), \quad \Phi_p(z) = 1 + z + \cdots + z^{p-1}. \quad (34)$$

This is a free L -module with basis $1, z, \dots, z^{p-2}$. It is *not necessarily a field*. For example, when L already contains ζ_3 ,

$$(z - \zeta_3)(z - \zeta_3^2) = 0 \quad (35)$$

in A , although neither factor is zero. Treating such a quotient as a field and inverting arbitrary nonzero elements would be a real correctness bug.

The implementation uses only addition, multiplication, and positive integer powers in A . Its coefficients remain elements of the exact field L . Reduction uses

$$z^{p-1} = -(1 + z + \cdots + z^{p-2}), \quad z^p = 1. \quad (36)$$

No tensor-field inversion or linear-disjointness test is needed.

8.2 The resolvent identities

For $0 \leq k < p$, define

$$R_k(z) = \sum_{j=0}^{p-1} z^{-kj} \sigma^j(\beta). \quad (37)$$

Negative powers can be reduced modulo p because $z^p = 1$. The action of H on A fixes the formal symbol z and acts on coefficients.

Proposition 8.1. *The following identities hold in the tensor algebra:*

$$R_0 \in L^H, \quad (38)$$

$$\sigma R_k = z^k R_k, \quad (39)$$

$$R_k^p \in L^H[z]/(\Phi_p(z)) \quad (1 \leq k < p), \quad (40)$$

$$p\beta = \sum_{k=0}^{p-1} R_k. \quad (41)$$

Proof. Normality of N implies that every conjugate $\sigma^j(\beta)$ is fixed by N . The relation $\sigma^p(\beta) = \beta$ makes a cyclic index shift in (37) valid and gives (39). Its p th power is fixed by σ and also by N , hence by H . Since the displayed basis of A is free over L , invariance of an element is equivalent to invariance of each coefficient; this proves (40) even if A has zero divisors. The same argument for $k = 0$ proves (38).

Finally, summing (37) over k gives $p\beta$ because $\sum_{k=0}^{p-1} z^{-kj} = 0$ for $1 \leq j < p$, and equals p for $j = 0$. For nonzero j , primality of p permutes the nontrivial exponents modulo p , so this vanishing is exactly the relation $\Phi_p(z) = 0$. $\square$

8.3 Recursive conversion

Write

$$R_k(z)^p = \sum_{\ell=0}^{p-2} q_{k\ell} z^\ell, \quad q_{k\ell} \in L^H. \quad (42)$$

The trace and these coefficients belong to the preceding fixed-field level and can be recursively converted. Once their radical expressions are known, evaluate $z = \zeta_p$ and form

$$Q_k = \sum_{\ell=0}^{p-2} \text{Rad}(q_{k\ell}) \zeta_p^\ell. \quad (43)$$

There is an integer $e_k \in \{0, \dots, p-1\}$ for which

$$R_k(\zeta_p) = \zeta_p^{e_k} Q_k^{1/p}. \quad (44)$$

The next section explains how that integer is selected without numerical guessing. The reconstructed expression is

$$\text{Rad}(\beta) = \frac{1}{p} \left[\text{Rad}(R_0) + \sum_{k=1}^{p-1} \zeta_p^{e_k} Q_k^{1/p} \right]. \quad (45)$$

At the bottom of the subgroup chain, the fixed elements are rational coefficient vectors. The recursion therefore terminates. Values already fixed by the preceding, larger subgroup skip a level, and equal coefficient vectors at the same level share a memoized result.

8.4 Zero resolvents are not exceptions to the theory

A resolvent may be the zero tensor, or a nonzero tensor may evaluate to zero at the chosen pair of embeddings. Neither case invalidates Fourier inversion. A zero summand contributes zero; it is never used as a denominator. The implementation distinguishes formal zero from zero in the chosen embedding when choosing branches. This is another advantage of using all the Fourier components to reconstruct the requested element rather than insisting that one particular component be a nonzero field generator.

9 Branch selection using an exact annihilator

A root-of-unity multiplier is not optional decoration. Independent principal p th roots of the Q_k generally do not reconstruct β . The branch choice must be tied to the concrete element whose field coordinates are known.

9.1 The norm polynomial

Represent $R_k(z)$ by $R(u, z)$ with rational coefficients reduced in u modulo h and in z modulo Φ_p . Compute

$$P_R(V) = \text{Res}_u(h(u), \text{Res}_z(\Phi_p(z), V - R(u, z))). \quad (46)$$

It is a monic rational polynomial of degree $d(p-1)$ before repeated factors are removed. It is the characteristic/norm polynomial of the multiplication operator by R in the finite algebra (34). It annihilates every evaluation of R at a conjugate of t and a primitive p th root of unity. The code uses its monic square-free part.

Lemma 9.1. *For $1 \leq k < p$, every candidate $\zeta_p^e Q_k^{1/p}$ in (44) is a root of P_R .*

Proof. If $R_k(\zeta_p) = 0$, all candidates are zero. Otherwise the p candidates are precisely the p values obtained by multiplying $R_k(\zeta_p)$ by a p th root of unity. Equation (39) rotates that value by ζ_p^k when σ acts on the L coordinates and fixes z . Because k is coprime to the prime p , its powers generate all these multipliers. All those evaluations occur among the roots of the norm polynomial. The equality $Q_k = R_k(\zeta_p)^p$ is already established by the exact tensor power and the induction hypothesis for its coefficients. $\square$

The proof is important: it justifies a common-annihilator root comparison for *both* expressions being compared. Passing an arbitrary numerical guess to such a comparison would not be justified.

9.2 Equality is easier when a polynomial is already known

For a square-free rational polynomial P , distinct complex roots have an effectively computable positive separation bound δ . If two known roots are evaluated with absolute error less than $\delta/8$, their approximate distance is less than $\delta/4$ when they are equal and greater than $3\delta/4$ when they are different. A threshold between these cases decides equality. The evaluation tolerance comes from a theorem about P , not from a user's expectation that 50 or 100 decimal places ought to be enough.

SymPy's `Poly.same_root` is specifically an equality test for two known roots [8]. In the pinned implementation it uses a Mignotte separation bound and bounded-absolute-error evaluation. The

package invokes it only after a structural identity, a factor relation, or Lemma 9.1 supplies the known-root premise. The degree-one case is handled separately, because under that premise there is only one root.

This remains a trusted-CAS calculation, not a formally verified interval kernel supplied with the package. In particular, the report does not claim that a small independent proof assistant has checked SymPy’s numerical evaluator. The distinction from heuristic recognition is that the polynomial and required separation are established algebraically before the comparison is made.

9.3 Two levels of verification

The general engine maintains construction invariants: exact field operations, fixed-field membership, the resolvent-power identity, and a valid common annihilator for each branch comparison. The resulting expression therefore inherits its algebraic identity from the construction.

An independent verifier can instead forget this history, compute the minimal polynomial m_E of the final radical expression E , check $m_E \mid f$, and then compare embeddings. The Python `verify` function does exactly that. It is useful for regression tests, but can be dramatically slower: treating correlated radicals independently may create a much larger temporary field. The C5 experiment in Section 13 illustrates this distinction.

The Wolfram frontend performs an additional check

```
RootReduce[answer - input] === 0
```

and returns no candidate unless it passes. A timed-out or unresolved check is not silently replaced by numerical proximity.

10 Correctness and finite completeness

Theorem 10.1. *Let $f \in \mathbb{Q}[x]$ be irreducible and let an embedding of one of its roots be specified. With resource bounds removed and correct terminating exact factorization, resultant, and algebraic-embedding primitives, the general algorithm terminates. It produces a radical expression for the specified root when that root is expressible by radicals, and otherwise proves nonsolvability through its Galois group.*

Proof. Section 5 constructs the splitting field in finitely many adjunctions, with every primitive-element search terminating by Lemma 5.1. Proposition 6.1 gives all automorphisms by a finite search and exact polynomial tests. The finite multiplication table then either detects a nontrivial perfect subgroup during the derived-subgroup construction or gives the prime-index chain (33).

In the former case G is not solvable, and the necessity direction of the Galois solvability theorem excludes a radical expression for any root of this irreducible polynomial. In the latter case use induction on fixed-field level. The base field is $L^G = \mathbb{Q}$. For the inductive step, Proposition 8.1 puts the trace and all coefficients of each resolvent power in the preceding level. The induction hypothesis converts them to radicals. There are finitely many possible branches, and Lemma 9.1 supplies a correct annihilator for a terminating algebraic equality test that selects the actual one. Fourier inversion then proves the result equals the original field element. The induction constructs all roots, and an exact comparison selects the requested embedding. $\square$

Corollary 10.2. *A resource cutoff during this calculation proves nothing about radical solvability. In particular, `ResourceLimit` and `NotSolvable` are logically different outcomes.*

The theorem does not require all extensions in the raw root-adjunction tower to be radical extensions. They usually are not presented that way. That tower is a device for computing the normal field and its action. The radical tower is built only afterward from its solvable group.

Nor does the theorem assert that every intermediate field has a primitive generator of lower absolute degree than the original input. Normal closures and cyclotomic coordinates can be larger than the original root field. Trying to force monotone decrease of the *absolute minimal-polynomial degree* at every discovery step is not a valid general completeness principle. What strictly decreases during radical synthesis is the subgroup-chain level.

11 Implementation details that affect correctness

11.1 Rational vectors and orientation

The reference implementation uses SymPy’s exact algebraic-number polynomial representation. Its low-level `ANP.to_list()` coefficients are in descending order, while tensor tuples are in ascending powers of z . The conversion helpers isolate that difference. The defining minimal polynomial is explicitly supplied to `QQ.algebraic_field`; it is not recomputed from an already complex nested expression.

Automorphism composition uses polynomial substitution, not arithmetic on approximate roots. Group elements are integers indexing the multiplication table. Subgroups are sets of these indices. Memoization keys for field elements are their exact coefficient tuples and their descent level; different conjugates are not conflated merely because their minimal polynomials agree.

The code accesses some SymPy representation-level interfaces, including `Poly.rep` and algebraic-number polynomial lists. The dependency is pinned to SymPy 1.14.0, the version used for testing. Porting to another representation or library should preserve these orientation and normalization invariants.

11.2 No arbitrary tensor division

Tensor multiplication forms the coefficient convolution and reduces terms from highest degree downward with $z^{p-1} = -\sum_{j=0}^{p-2} z^j$. Exponentiation uses repeated squaring. At no point does the code ask for the inverse of a nonzero tensor element. Field-element inverses, such as those needed for exact linear factors, occur only in the genuine field L . A regression test explicitly multiplies two nonzero zero divisors in the $p = 3$ tensor algebra and checks that the product is zero.

11.3 Expression DAGs, not executable source strings

The Python backend serializes output as a directed acyclic graph. Its node tags are `Q`, `I`, `Add`, `Mul`, and `Pow`. Rationals carry signed decimal numerator and denominator strings; references are zero-based indices of earlier nodes. The root index points to the final value. Every power exponent must decode to a rational number.

The Wolfram decoder validates tags, arities, rational denominators, backward references, and the final radical grammar. It uses integer parsing and arithmetic constructors, not `ToExpression`. The Python CLI similarly accepts rational coefficient data rather than expressions for `eval`. This also avoids copying repeated subexpressions into the interchange format. It does not guarantee that every later CAS pretty-printer will preserve sharing when displaying the full formula.

11.4 Root indices are not portable

The Python API defines `index` by SymPy's `CRootOf(p, index)` ordering, including multiplicities when a selected root is requested. Its `all_roots` API returns distinct roots with no promised ordering.

The Wolfram frontend never translates a Mathematica root index into a Python root index. It sends the minimal polynomial, requests all its roots, and selects a candidate by exact equality to the original Wolfram expression. The extra work buys a simple and reliable cross-system embedding contract. A more optimized future protocol could transmit an exact isolating rectangle instead, but approximate coordinates alone would not justify a change in this contract.

11.5 Failure and time-limit semantics

The important statuses are shown in Table 1. Python exposes status-bearing exception classes; the CLI serializes analogous strings; the Wolfram frontend uses `Failure` expressions.

Status	Meaning
<code>Success</code>	A radical expression was constructed with the method's checks.
<code>NotSolvable</code>	A nonsolvable Galois group was established for the relevant irreducible factor.
<code>ResourceLimit</code>	A field-degree budget or computation deadline was exceeded.
<code>InvalidInput</code>	Input was not in the exact rational-polynomial contract, or options/data were malformed.
<code>Unresolved</code>	The explicitly requested native-only structural search did not yield a verified result.
<code>BackendError</code>	An underlying computation failed; no mathematical impossibility follows.
<code>VerificationFailed</code>	The frontend did not establish equality to the requested embedding.

Table 1: Operational outcomes must not be collapsed into nonsolvability.

The in-process Python `max_seconds` budget is cooperative: it is checked between operations, so a single factorization or root-isolation call can run past the deadline. The bundled `python/cli.py` additionally supports `hard_timeout_seconds`; it starts an isolated worker and terminates that worker when the deadline expires. The Wolfram frontend uses that supervisor when its `TimeConstraint` is finite.

There is still no single global Wolfram deadline: native candidate generation, the Python worker, and final per-candidate exact verification have separate stages. The last stage is controlled by `"VerificationTimeConstraint"`; its default is `Infinity`. These semantics are intentional and documented rather than hidden behind a misleading total-time promise.

12 Using the package

12.1 Wolfram Language

Keep the unpacked directory layout intact and load the file:

```
Get["C:\\path\\to\\RadicalSolve\\wolfram\\RadicalSolve.wl"];
r = Root[-1-#^2-#^3+#^4+#^6&, 2];
a = Radicalize[r, Method -> "Native"];
VerifyRadical[r, a]
RadicalReport[r, Method -> "Native"]
```

The native route includes the structural methods and low-degree `ToRadicals` formulas. Both original examples have native structural routes, so this use does not require Python.

For the general route, install the pinned Python dependency using the chosen interpreter, then supply its executable name or absolute path:

```
RadicalReport[Root[#^3 - 3# + 1&, 1],
  Method -> "Galois",
  "PythonExecutable" -> "python",
  "MaxFieldDegree" -> 96,
  TimeConstraint -> 120,
  "VerificationTimeConstraint" -> 120]
```

`Method -> Automatic` tries native methods first and then the Python backend. `Method -> "Native"` never launches it. `Method -> "Galois"` bypasses structural reduction for non-radical input. An expression already in the radical grammar is simply returned.

`RadicalReport` returns an association containing the expression, method, and verification result. Backend construction metadata is included when applicable. The default field-degree cap is 96; use `Infinity` to remove it.

12.2 Python

From the archive root, install or use the unpacked module:

```
python -m pip install -e .
python -m unittest discover -s tests -v
```

Alternatively add the `python/` directory to `PYTHONPATH`; no package installation is required for the bundled CLI.

```
from sympy import symbols, Poly
from radicalsolve import Solver, radicalize, verify
x = symbols("x")
p = Poly(x**6+x**4-x**3-x**2-1, x)
r = radicalize(p, index=1)
assert verify(p, r.expression, index=1)
print(r.expression)
print(r.method)
```

```

session = Solver(method="galois", max_field_degree=96)
q = Poly(x**5+x**4-4*x**3-3*x**2+3*x+1, x)
roots = session.all_roots(q)
print(session.events)

```

`max_field_degree=None` removes the Python degree cap. `Solver.root` first selects the irreducible factor containing the requested root. This is why it can return a solvable root of $(x^2 - 2)(x^5 - x - 1)$ without being blocked by the nonsolvable quintic factor. In contrast, `all_roots` requires all irreducible factors to be solvable.

`Result.expression` is the result, `Result.method` is the route, and `Result.record` is an audit record. The record is *not* a standalone, independently replayable proof certificate. This version performs its construction checks during execution and offers independent minimal- polynomial verification separately. A compact portable certificate checker is an explicit future engineering improvement, not an already supplied feature.

13 Executed tests and observed limitations

13.1 Environment and reproducibility

The reference code was executed with Python 3.13 and SymPy 1.14.0. The archive contains the actual `unittest` transcript and a separate general- S_4 stress transcript. The declared Python requirement is 3.10 or later; only the stated environment was exercised here.

No usable Wolfram kernel was available. Both attempted Wolfram connector calls failed before evaluation. Consequently the supplied `.wl` and `.wlt` files have source-level review but *no claimed Wolfram runtime test*. The Python engine is the executed reference, and the native formulas are also justified mathematically in this report.

13.2 Coverage

The default suite exercises the two original selected roots with independent minimal-polynomial and embedding checks; it also matches every conjugate from the two structural constructions. Additional tests cover translated inputs, power compositions, rational roots, multiplicities, a mixed reducible solvability example, wrong conjugates, invalid coefficients, invalid indices, tensor zero divisors, data encoding, and process deadlines.

The forced-general-method families are listed in Table 2. These tests bypass the fast structural layer; otherwise, examples such as $x^4 - 2$ would say nothing about the general engine.

The C_5 example is the real cyclotomic degree-five polynomial associated with order eleven. It can be derived by dividing $1 + z + \cdots + z^{10}$ by z^5 and expressing the symmetric terms in $x = z + z^{-1}$. This gives a useful independent mathematical description of its solvability, although the test forces the implemented Fourier route rather than using that description as a shortcut.

The default test run reports the actual pass/skip count in the included transcript. The optional forced- S_4 integration test is skipped by default. The ordinary automatic/classical route for a generic quartic is nevertheless included and passes its tests.

Group	Polynomial	$[L : \mathbb{Q}]$	Independent final check
C_2	$x^2 + 1$	2	minimal polynomial
C_3	$x^3 - 3x + 1$	3	minimal polynomial
S_3	$x^3 - 2$	6	minimal polynomial
V_4	$x^4 - 10x^2 + 1$	4	minimal polynomial
C_4	$x^4 + x^3 + x^2 + x + 1$	4	minimal polynomial
D_4	$x^4 - 2$	8	minimal polynomial
C_5	$x^5 + x^4 - 4x^3 - 3x^2 + 3x + 1$	5	numerical residual only

Table 2: Executed general-descent families. All rows also execute the internal exact algebraic invariants and separation-based branch checks. “Numerical residual only” describes the additional, independent final check, not the construction algorithm. D_4 denotes the group of order eight.

13.3 What the stress run did and did not establish

For $x^4 - x - 1$, a forced-general-method exploratory run constructed the degree-24 splitting field, recovered its automorphisms from weights $(1, 2, 3)$, and obtained the chain of subgroup orders

$$24, 12, 4, 2, 1. \quad (47)$$

The corresponding prime indices were 2, 3, 2, 2. That run did not complete radical output within its cooperative time budget. A sampled stack was inside complex root isolation used by a branch comparison. It is recorded as a resource-limited run, not as a passing full integration test.

For the cyclic quintic, constructing all five radical expressions in an exploratory run took about a second in this environment. A separate attempt to reconstruct the minimal polynomial of the expanded result ran much longer and was stopped. The completed regression test therefore uses the construction checks and an additional high-precision residual rather than claiming that the independent reconstruction finished. This is an observed verification-cost issue, not a claim of a mathematical ambiguity in the formula.

The large inverse-beta-related notebook examples, including degrees beyond those in Table 2, are retained as source material. They were not silently promoted into a “passed” benchmark set. A useful next benchmark campaign would compare normal-closure degree, coefficient growth, descent size, and verification time on those exact inputs.

14 Complexity and further improvements

14.1 The main costs

Write $d = [L : \mathbb{Q}] = |G|$. The input degree n alone is a poor predictor of cost: d can approach $n!$. A full multiplication table requires $O(d^2)$ entries, and constructing it involves polynomial substitutions in a degree- d field. The primitive-image search examines at most $n!/(n-r)!$ ordered tuples when r roots occur in the primitive sum.

At a cyclic step of prime index p , tensor coordinates have dimension $p-1$ over L . Their rational norm polynomial has degree $d(p-1)$ before square-free reduction. In a naive recursion, a level can create up to $(p-1)^2$ coefficient subproblems plus a trace subproblem. Memoization helps substantially but does not turn this into a general polynomial-time output algorithm. Rational coefficient heights and algebraic-number isolation costs are equally important.

The resulting formula need not be close to minimal. A short radical can acquire a large expression because the construction used a complicated primitive coordinate system. The structural layer exists precisely to avoid that unnecessary detour whenever an exact cheap identity is available.

14.2 Improvements that preserve the proof structure

The most useful extensions are computational, not changes to the solvability criterion. Better number-field factorization and root-isolation backends would address the observed bottlenecks. Smaller primitive elements and reduced bases could limit coefficient growth. Acting with subgroup generators rather than checking all subgroup elements could reduce repeated fixed-field tests after an exact generating set has been certified.

A more compact descent can choose a suitable nonzero eigenvector and a relative power basis, then express several requested elements in that basis. That may share radicals more effectively than reconstructing each element from all Fourier components. It introduces extra nonvanishing and basis-independence obligations, especially when a cyclotomic tensor splits. The current version accepts the larger formulas in exchange for a simple division-free argument.

The native structural recognizers could also be broadened to rational-function decompositions, additional resolvent families, or systematically certified subfield searches inspired by the notebook. Those methods should remain optimizations: a failed structural search must not overwrite the general solvability status.

Finally, a portable proof record should include the field polynomial, automorphism images, subgroup chain, resolvent-power identities, and selected branch witnesses. An independent checker could then replay exact identities without rediscovering a large minimal polynomial from a flat formula. The present audit metadata is not yet that full object.

15 Conclusion

A systematic converter needs more than a longer list of radical formulas. It needs the normal field and its actual action, a descent invariant that truly decreases, and an embedding contract that survives every radical extraction. The supplied implementation gives those pieces concrete representations.

The two motivating examples receive short structural answers. The general engine implements a finite solvable-Galois construction, including prime cyclic steps beyond cubic and quartic formulas. The tensor formulation avoids assuming that roots of unity are disjoint from the splitting field, and the norm argument makes branch comparisons legitimate before any numerical evaluation is used. The delivered code is a tested reference implementation with explicit resource and validation boundaries, rather than a claim that every large solvable input will already have an economical answer.

A Compact algorithm descriptions

A.1 Normal closure and automorphisms

```

h := irreducible input polynomial f
weights := [1]
repeat:
  K := Q[t]/h
  factor f over K
  if all factors are linear: break
  choose a nonlinear irreducible factor g of degree e
  for c = 1, 2, ...:
    H := Res_u(h(u), c^e g((T-u)/c, u))
    if degree(H) = e degree(h) and H is square-free:
      h := monic(H)
      append c to weights
      break

roots := linear roots of f in K
images := distinct weighted sums of distinct roots satisfying h(image)=0
require number(images) = degree(h)
construct automorphism multiplication by exact substitution

```

A.2 One radical-conversion call

```

Rad(beta, level):
  require beta fixed by H[level]
  if beta is rational: return beta
  if beta fixed by H[level-1]: return Rad(beta, level-1)
  p, sigma := current prime quotient and generator
  pieces := [Rad(trace(beta), level-1)]
  for k = 1, ..., p-1:
    R := sum_j z^(-k j) sigma^j(beta) modulo Phi_p(z)
    if R is formally zero: append 0; continue
    P := square-free rational norm polynomial of R
    handle a linear P or zero in the selected embedding directly
    q := R^p, computed without division in the tensor algebra
    require every coefficient of q fixed by H[level-1]
    Q := evaluate recursively converted q at z = zeta_p
    select e with R(zeta_p) = zeta_p^e Q^(1/p)
    using the common polynomial P and root separation
    append zeta_p^e Q^(1/p)
  return sum(pieces)/p

```

B Files and rebuilding

The archive's top-level `README.md` gives installation and command examples. The implementation files are under `python/radicalsolve/`; the direct supervised entry point is `python/cli.py`. The native Wolfram frontend is `wolfram/RadicalSolve.wl`. Tests and actual transcripts are under `tests/`. The unchanged original notebook and its source hashes are under `provenance/`.

The article can be rebuilt in its directory with

```
latexmk -pdf radicalsolve.tex
```

It uses ordinary LaTeX packages and Latin Modern fonts. No external images, commercial fonts, or downloaded font files are required. New implementation and report material are provided under MIT-0; the original notebook and third-party dependencies retain their existing rights.

References

- [1] Vladimir Reshetnikov. *Why is ToRadicals not able to handle all cases? Is there a workaround?* Mathematica Stack Exchange, question 34011 and associated discussion. <https://mathematica.stackexchange.com/questions/34011/>. Accessed 8 September 2026.
- [2] Vladimir Reshetnikov. *FindExtension.nb*, supplied in **FindExtension.zip**, received 8 September 2026. Unchanged source and SHA-256 hashes included in the archive.
- [3] J. S. Milne. *Fields and Galois Theory*, version 5.10, 2022. Especially the cyclic-extension and solvability material in Chapter 5. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [4] Song Li. *An Algorithm for Solving Solvable Polynomial Equations of Arbitrary Degree by Radicals*. 2022. arXiv:2203.11602. <https://arxiv.org/abs/2203.11602>.
- [5] Wolfram Research. *ToRadicals*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToRadicals.html>. Accessed 8 September 2026.
- [6] Wolfram Research. *ExactNumberQ*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ExactNumberQ.html>. Accessed 8 September 2026.
- [7] SymPy Development Team. *Number Fields*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/numberfields.html>. Accessed 8 September 2026.
- [8] SymPy Development Team. *Polynomials Manipulation Module Reference*, including `Poly.same_root`, SymPy 1.14.0 documentation and implementation. <https://docs.sympy.org/latest/modules/polys/reference.html>. Accessed 8 September 2026.